EECS 348 Group 10

Arithmetic Expression Evaluator
Software Architecture Document

Version <1.0>

Revision History

Date	Version	Description	Author
4/9/26	<1.0>	Finished section 5	Jaycob Campos
4/12/26	<1.0>	Completed Sections 6, 7, 8	Tate Meyer
4/12/26	<1.0>	Completed Section 3	Avery Richardson
4/12/26	<1.0>	Section 1(1.1, 1.2, 1.3)	Nikki Sotoodi
4/12/26	<1.0>	Section 2, 1.5	Maddi Clark

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4
2. Architectural Representation 5
3. Architectural Goals and Constraints 5
4. Use-Case View 5
   - 4.1 Use-Case Realizations 5
5. Logical View 5
   - 5.1 Overview 5
   - 5.2 Architecturally Significant Design Packages 5
6. Interface Description 8
7. Size and Performance 8
8. Quality 9

Software Architecture Document

1. Introduction

The Arithmetic Expression Evaluator is a software system designed to process and evaluate arithmetic expressions provided by the user. The system takes an expression as input, analyzes its structure, and computes the correct result based on standard mathematical rules.

1.1 Purpose

This document provides a high-level overview of the software architecture for the Arithmetic Expression Evaluator system. It describes the structure of the system, the main components, and how they interact to process and evaluate arithmetic expressions.

The purpose of this document is to communicate the key architectural decisions made during the design of the system and to serve as a reference for developers, instructors, and team members. It is intended to help guide implementation, testing, and future modifications of the system.

1.2 Scope

This document applies to the Arithmetic Expression Evaluator system developed for EECS 348. The system is designed to take an arithmetic expression as a string input, process it through multiple stages (tokenizing, parsing, and evaluation), and return either a computed result or an error message.

The document focuses on the overall system architecture, including the major modules, their responsibilities, and their interactions. It does not include low-level implementation details or specific code.

1.3 Definitions, Acronyms, and Abbreviations

Token: A basic unit of the expression, such as a number, operator, or parenthesis.
Parser: A component that analyzes tokens and builds a structured representation (tree).
Node: An element in the syntax tree representing a value or operator.
PEMDAS: Order of operations (Parentheses, Exponents, Multiplication/Division, Addition/Subtraction).
CLI: Command Line Interface.
Expression: A combination of numbers, operators, and parentheses that can be evaluated.

1.4 References

- UPEDU's Software Architecture Document template

1.5 Overview

This document is organized into various sections. This includes a section on architectural representation, architectural goals and constraints, use-case view, logical view, an interface description, size and performance of the software, and the quality of the software. Each of these sections may or may not have subsections to break down information further.

2. Architectural Representation

This document present the architecture as views, specifically the logical view. There is no use-case view described.

3. Architectural Goals and Constraints

3.1 Architectural Goals:

Ensure correct parsing of arithmetic expressions according to the defined rules
Implement robust error detection and reporting for invalid syntax or incorrect expressions.
The program maintains high performance for all expressions.
Support extensibility for any potential future (i.e. more complex functions)

3.2 Architectural Constraints

Must be implemented in C++
Ensure the program cannot crash on malformed input and must avoid any undefined behavior.
The program must support only arithmetic expressions within the project scope.
The program must take input in only as a string, and output must be either a numeric result or a error that the user can resolve.

4. Use-Case View

4.1 Use-Case Realizations

5. Logical View

The system is decomposed into four modules: Tokenizer, Parser, Evaluator, and Error Handler. The Tokenizer creates Token objects and passes them as a list to the Parser. The Parser uses those Token objects to build a tree of Node objects. The Parser passes the root Node to the Evaluator, which recursively traverses the tree to compute the final result. All three stages call into the Error Handler when errors are encountered. Detailed descriptions of each module and their classes are provided below.

5.1 Overview

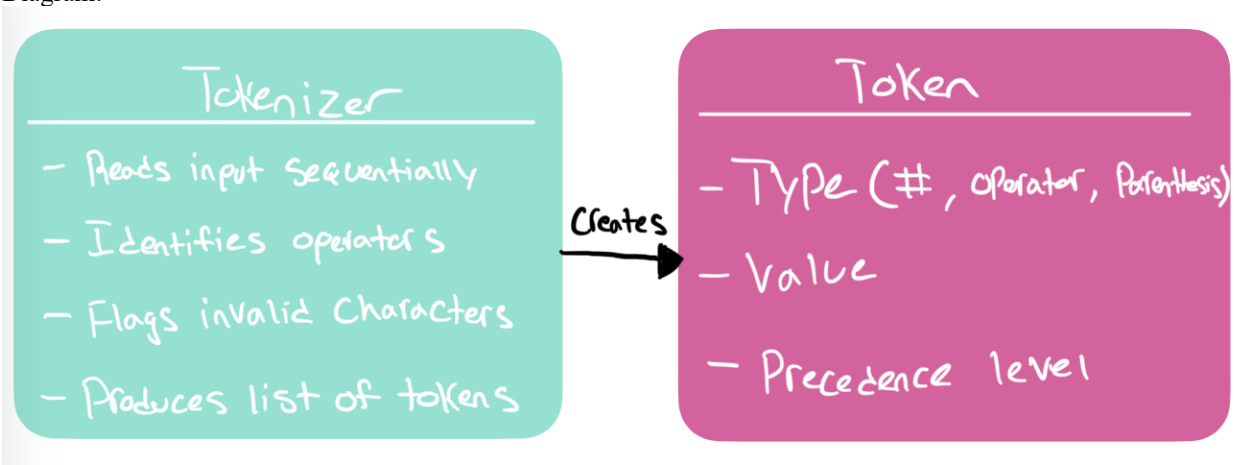
The arithmetic parser will use a pipe and filter architecture with four modules. The first module is a tokenizer that takes in raw input and breaks it into tokens that hold necessary data for evaluating. The second module is a parser that takes those tokens and puts them into a tree where the operators are the parents, and the operands are the children. The ordering process for the parents will follow left-to-right and right-to-left associativity. Parentheses are handled by skipping them when selecting the root, then making their contents a child subtree to maintain PEMDAS. The third module is the evaluator, which recursively evaluates the tree from the root down, resolving children before computing each parent. The fourth module is the Error Handler, which provides centralized error reporting across all stages. The tokenizer catches invalid characters and unsupported operators, the parser catches mismatched parentheses and structural issues, and the evaluator catches runtime errors like division by zero. Each stage detects errors and reports them through the Error Handler.

5.2 Architecturally Significant Design Modules or Packages

Tokenizer Module

The Tokenizer module is responsible for converting raw input into a list of tokens for the Parser. The module will contain a token class and a tokenizer class. The Token class is a data container that holds information about a single element of the expression. Its attributes are type (Number, Operator, Parenthesis), Value, and precedence level. The tokenizer class takes the raw input string and creates token objects by reading through the input and separating into individual tokens. It reads characters sequentially, skipping whitespaces, and flagging any unsupported characters or operators as errors. It will produce a list of Token objects for the parser.

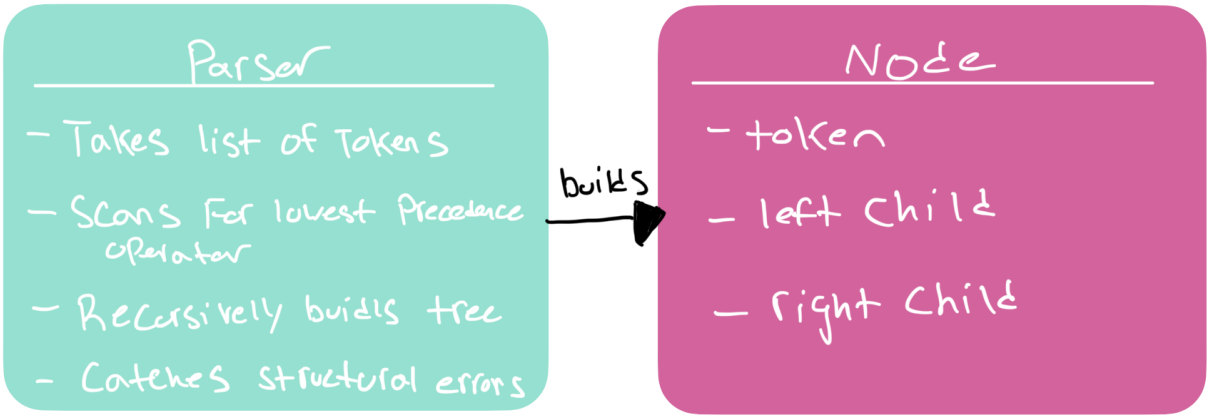
Diagram:



Parser Module

The Parser module is responsible for converting a list of tokens into a tree for the evaluator. The module contains a node class and a parser class. The node class is a data container that holds a token object, a left child node pointer, and a right child node pointer. The parser class takes a list of token objects and returns the root node of the tree. It builds the tree by scanning for the lowest precedence operator, not inside parentheses, selecting it as the root based on associativity rules. It then will recursively build the left and right subtree. When the parser encounters a parenthesized subexpression, it strips the parentheses and recursively builds a subtree from the contents. The parser also catches errors such as mismatched parentheses and missing operands during tree construction.

Diagram:



Evaluator Module

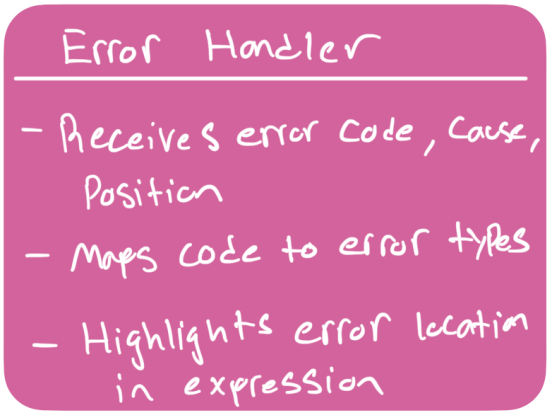
The Evaluator module is responsible for taking a syntax tree and computing the result. The module contains an Evaluator class. The Evaluator class takes in the root node of the tree and returns the final computed result. It works by recursively evaluating each node. If a node is a number, it returns its value. If a node is an operator, it evaluates both left and right children then applies the operation. The Evaluator will catch runtime errors such as division by zero.

Diagram:

Error Handler Module

The Error Handler module is responsible for reporting errors to the user across all stages of the pipeline. The module contains an Error Handler class. The Error Handler class receives an error code, the cause of the error, and the position in the expression where it occurred. It maps error codes to predefined types such as invalid character, unsupported operator, mismatched parentheses, missing operand, and division by zero.

Diagram:



6. Interface Description

The system operates as a command-line application. On launch, it prompts the user to enter an arithmetic expression as a string. It then displays either the computed result or a descriptive error message, then prompts again, looping until the user exits. No graphical interface is included in the implementation.

6.1 Input Interface:

- Accepts: integers, floats, operators (+, -, *, /, %, **), unary (+ -), parentheses, and white space.
- Invalid characters trigger a TOKEN_ERROR via the Error Handler with a position indicator.

6.2 Module Interfaces (Internal API)

Module	Input	Output
Tokenizer	Raw input string	List<Token>
Parser	List<Token>	Root Node*
Evaluator	Root Node*	Numeric Result (float/int)
Error Handler	Error code, cause, position	Formatted error message to stdout

6.3 Output Interface:

- On success: Prints the numeric result.
- On error: prints a human-readable message identifying the error type and position.

7. Size and Performance

7.1 Size

The application is intentionally lightweight. The four modules consist of a small number of classes. The recursive tree structure scales linearly with expression length. An expression of n tokens produces at most n modes. Memory usage is transient and negligible on any modern machine.

7.2 Performance

The tokenizer must process source code at ~100,000 lines/second on standard hardware. Given that the recursive descent parser and tree evaluator both run in O(n) time relative to the number of tokens, the system easily meets this

threshold for typical expressions. No performance bottlenecks are anticipated. The recursion depth is bounded by expression complexity; deeply nested parenthetical expressions could approach stack limits on unusual inputs, but this is not a concern for the stated scope.

8. Quality

8.1 Reliability

The system must never crash on malformed input. Every stage (Tokenizer, Parser, Evaluator) routes errors to the centralized Error Handler rather than terminating. The user is allowed to correct and re-enter expressions.

8.2 Extensibility

The pipe-and-filter architecture cleanly separates concerns. Adding new operators requires changes only to the Tokenizer and Evaluator with minimal impact to the parser tree structure

8.3 Portability

The program compiles under and C++ 17 compliant compiler on standard personal computers and KU lab machines. No platform-specific libraries or hardware dependencies exist.

8.4 Maintainability

Each module is independently testable. The Error Handler's centralized error code mapping makes it straightforward to add or modify error messages without touching pipeline logic.

8.5 Correctness

The architecture enforces PEMDAS through the Parser's precedence-aware root selection and the Evaluator's recursive traversal. Left-to-right associativity for +. -. *, /, % and right-to-left for ** are structurally encoded in the tree-building logic.